
Présentation du projet

Problématique

Dans le contexte actuel de transition énergétique et de développement des maisons intelligentes, la gestion efficace de la consommation électrique est devenue une nécessité.

Les prises classiques ne permettent pas de contrôler à distance les appareils branchés ni de mesurer leur consommation. Cela entraîne un gaspillage d'énergie et un manque de visibilité pour l'utilisateur. L'émergence de l'Internet des Objets (IoT) offre de nouvelles solutions pour rendre les équipements électriques plus intelligents et connectés.

C'est dans ce cadre que s'inscrit ce projet, dont la problématique principale est la conception et la réalisation d'une **smart plug** capable à la fois de mesurer la consommation électrique et d'être commandée à distance via une interface simple et conviviale.



Figure 7 : Prise Intelligent

Objectif du projet

L'objectif de ce projet est de concevoir, simuler, réaliser et tester une prise connectée intelligente, basée sur un microcontrôleur ESP32. Cette prise doit permettre à l'utilisateur :

- d'activer ou de désactiver un appareil électrique branché, à distance,
- de mesurer en temps réel la puissance et la consommation électrique,
- de transmettre les données via un protocole IoT (MQTT) vers un serveur de communication (HiveMQ),
- et enfin de visualiser et commander la prise grâce à une interface graphique développée sur Node-RED.

En atteignant ces objectifs, le projet contribue à l'amélioration du confort de l'utilisateur, à la réduction de la consommation énergétique et à la promotion de solutions domotiques accessibles.

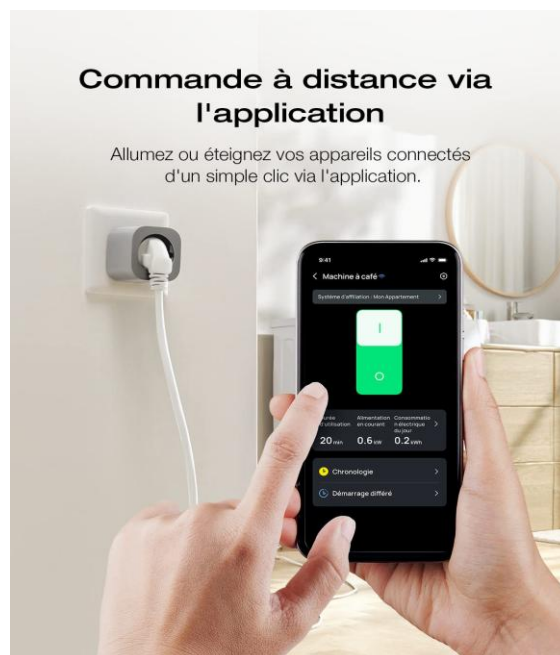


Figure 8 : Commande à distance

Méthodologie de travail

Afin de mener à bien ce projet, une méthodologie progressive a été adoptée.

Dans un premier temps, l'environnement de développement a été mis en place en installant Proteus pour la conception et la simulation des schémas électroniques.

Une étude préliminaire a permis de sélectionner les composants nécessaires (ESP32, relais, capteur de courant, alimentation, etc.), suivie de leur acquisition.

Ensuite, une phase de tests individuels a été réalisée pour vérifier le bon fonctionnement de chaque composant.

La deuxième étape a consisté à développer et tester le code de l'ESP32, intégrant la gestion du relais, la connexion Wi-Fi et la communication avec le broker MQTT (HiveMQ).

Enfin, une interface graphique a été conçue sur Node-RED afin de permettre à l'utilisateur de piloter la smart plug et de consulter les données en temps réel.

Cette approche méthodologique, organisée par étapes successives, a permis de progresser de manière structurée vers l'aboutissement du projet.

Conception et simulation en proteus

installation de l'environnement de travail

1. VirtualBox sur Ubuntu

Pour pouvoir exécuter Proteus, qui n'est pas compatible avec Linux, j'ai d'abord installé **VirtualBox** sur mon système Ubuntu. Cet hyperviseur m'a permis de créer une machine virtuelle dans laquelle je pouvais installer un système d'exploitation Windows.

2. Installation de Windows 10 sur VirtualBox

Une fois VirtualBox installé, j'ai créé une **machine virtuelle dédiée** en choisissant Windows 10 comme système invité. J'ai alloué la mémoire RAM, le stockage et les processeurs nécessaires, puis j'ai installé Windows 10 à partir d'un fichier ISO. L'installation s'est déroulée comme sur un ordinateur physique, et Windows était pleinement fonctionnel dans l'environnement virtuel.

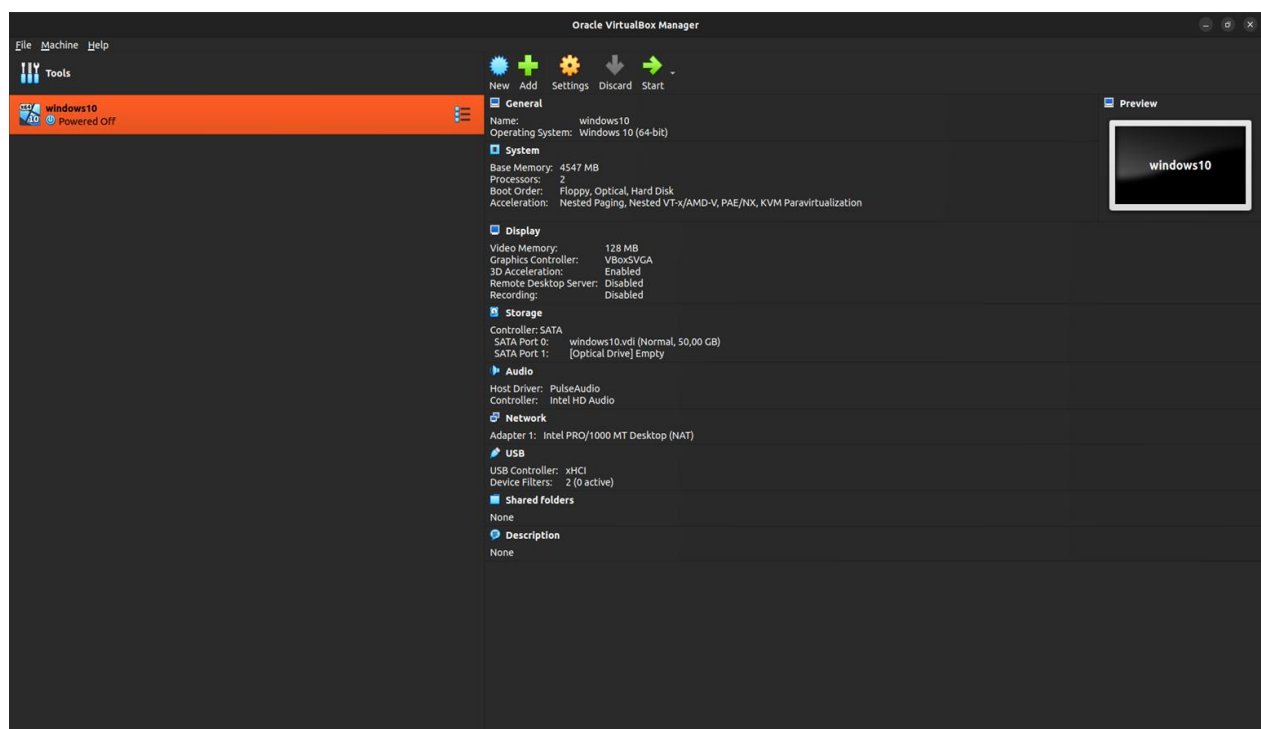


Figure 9 : Windows sur virtual box

3.Installation de Proteus sur Windows 10

Après avoir mis en place Windows 10 dans la machine virtuelle, j'ai installé le logiciel **Proteus**. L'installation s'est faite normalement via l'installateur de Proteus, et une fois terminée, j'ai pu lancer des simulations électroniques directement dans l'environnement Windows hébergé sur VirtualBox.

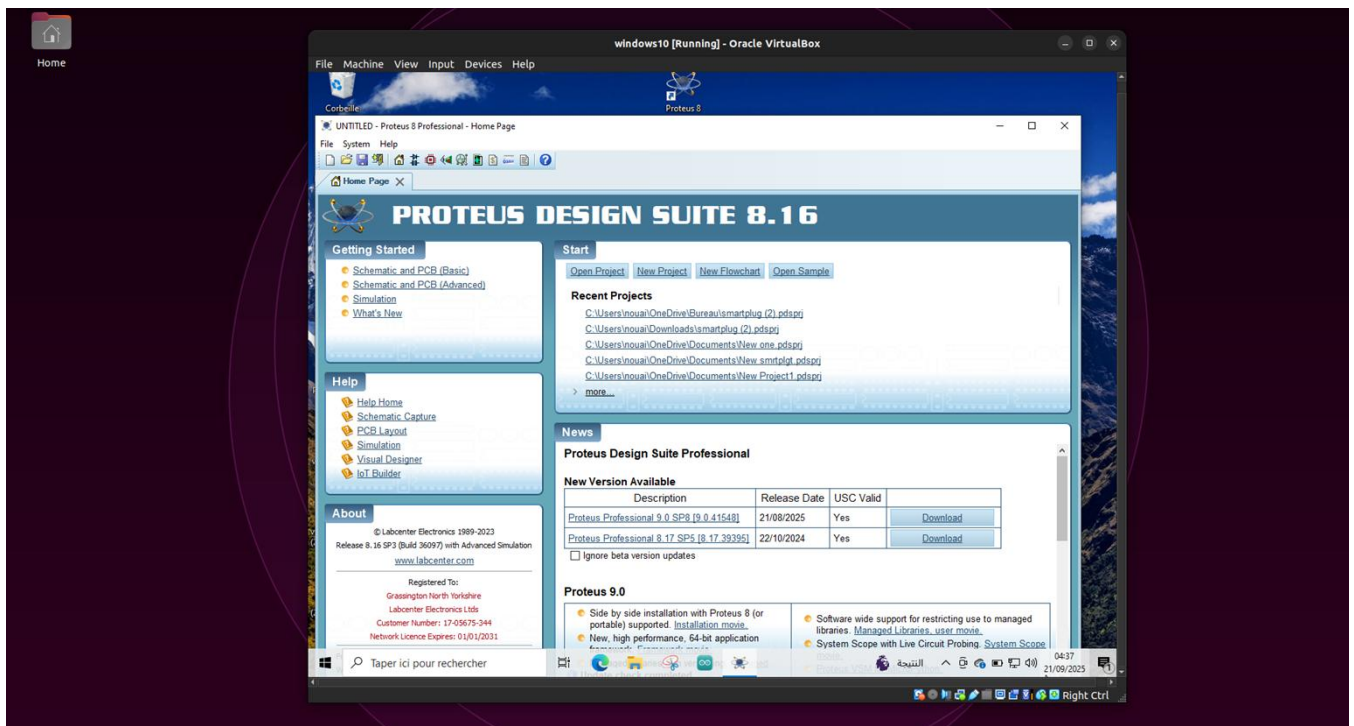


Figure 10 : Interface Proteus

Schémas électronique dans PROTEUS

Le schéma électronique du Smart Plug a été conçu et simulé sous Proteus afin de valider la faisabilité du montage avant la réalisation physique. Il est composé de plusieurs blocs fonctionnels interconnectés :

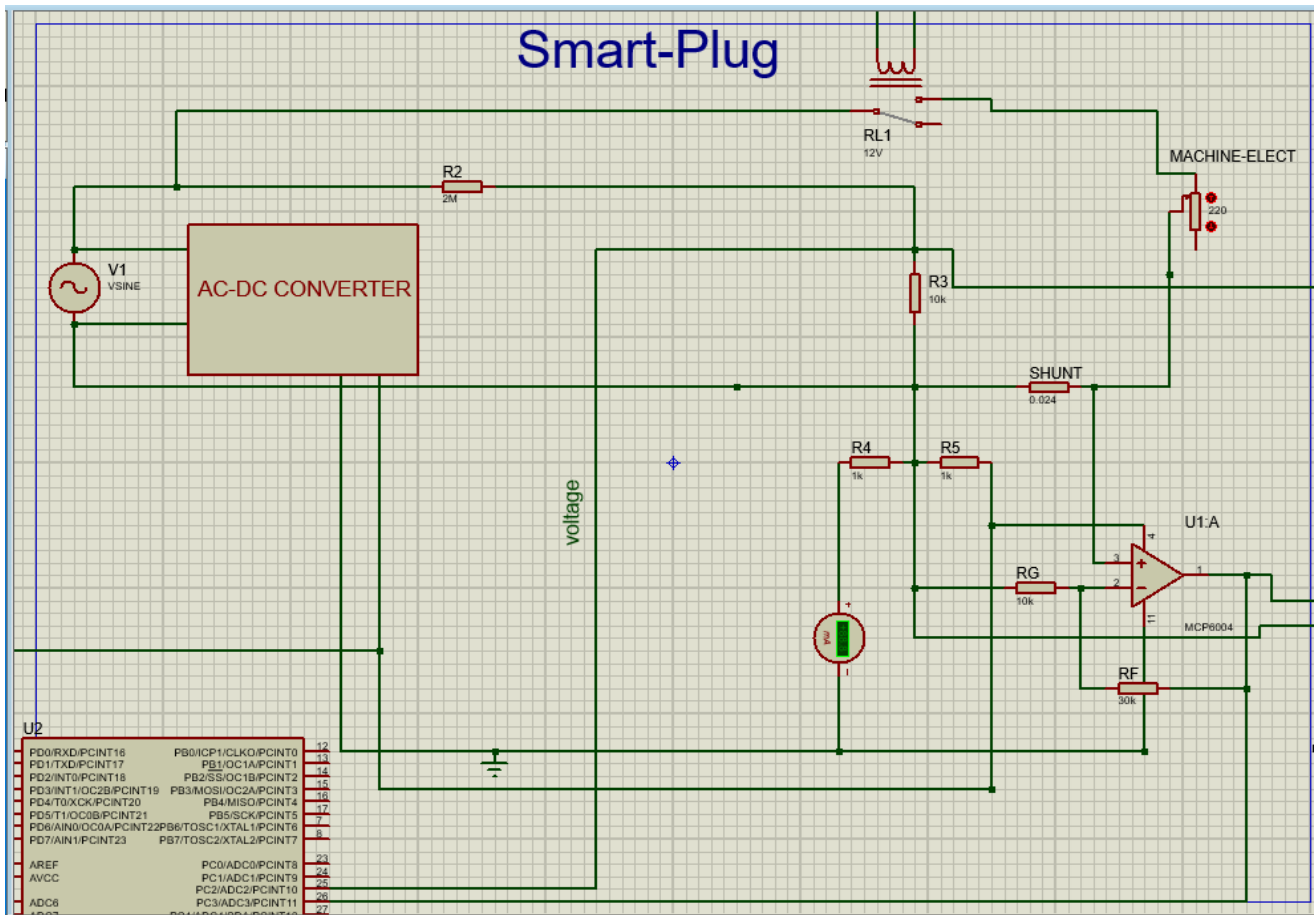


Figure 11 : Schéma électronique de smart plug

Bloc d'alimentation (AC-DC Converter) :

Ce bloc permet de convertir la tension alternative du secteur en une tension continue adaptée pour alimenter les différents circuits du montage. Il assure également une isolation et une protection des composants sensibles.

Bloc de commande (Relais)

Un relais 12V est utilisé pour contrôler l'alimentation de la charge (appareil électrique branché sur la prise). Ce relais est commandé par le microcontrôleur via un circuit d'interface. Il permet de mettre en marche ou d'éteindre l'appareil connecté de façon sécurisée

Charge connectée (Machine-Elect)

La charge symbolise l'appareil électrique branché sur la prise. Elle est reliée au secteur à travers le relais et le circuit de mesure

Bloc de mesure du courant (Shunt+aop)

Un résistor de shunt est placé en série avec la charge afin de générer une chute de tension proportionnelle au courant consommé. Comme cette tension est très faible, elle est amplifiée par un amplificateur opérationnel (MCP6004). Le signal amplifié est ensuite dirigé vers le microcontrôleur pour le calcul du courant absorbé par la charge.

Bloc de mesure de la tension (pont diviseur résistif)

La tension secteur est réduite par un pont diviseur de résistances (R_2 , R_3 , etc. dans ton schéma) afin de l'adapter au niveau d'entrée du microcontrôleur. Cette réduction est indispensable, car la tension du secteur (230 V) est trop élevée pour être appliquée directement sur un composant électronique. Le signal obtenu permet au microcontrôleur de mesurer et d'estimer la valeur efficace de la tension d'alimentation appliquée à la charge.

Bloc d'adaptation du signal pour l'ADC

Pour permettre une acquisition correcte des signaux alternatifs par le microcontrôleur un décalage continu (offset) a été introduit à l'aide d'une tension de référence égale à 1,65 V (moitié de 3,3 V).

En effet, le microcontrôleur ESP32 fonctionne avec une alimentation de 3,3 V et ses entrées analogiques ne peuvent pas mesurer des tensions négatives. Or, les signaux de courant et de tension issus du secteur sont de nature alternative et donc centrés autour de 0 V.

En ajoutant ce point milieu, le signal mesuré oscille désormais entre 0 V et 3,3 V au lieu de -1,65 V à +1,65 V. Cette méthode garantit que l'ensemble des valeurs lues par l'ADC (convertisseur analogique-numérique) du microcontrôleur soient toujours positives et compatibles avec sa plage d'entrée.

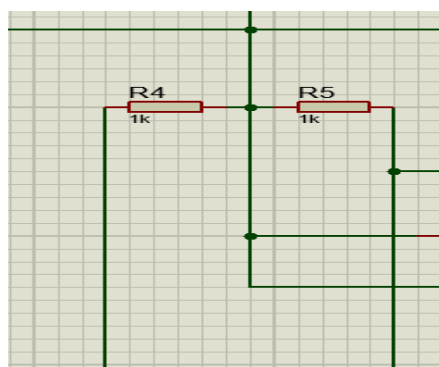


Figure 12 : point milieu

Simulation dans PROTEUS

1 ère simulation :

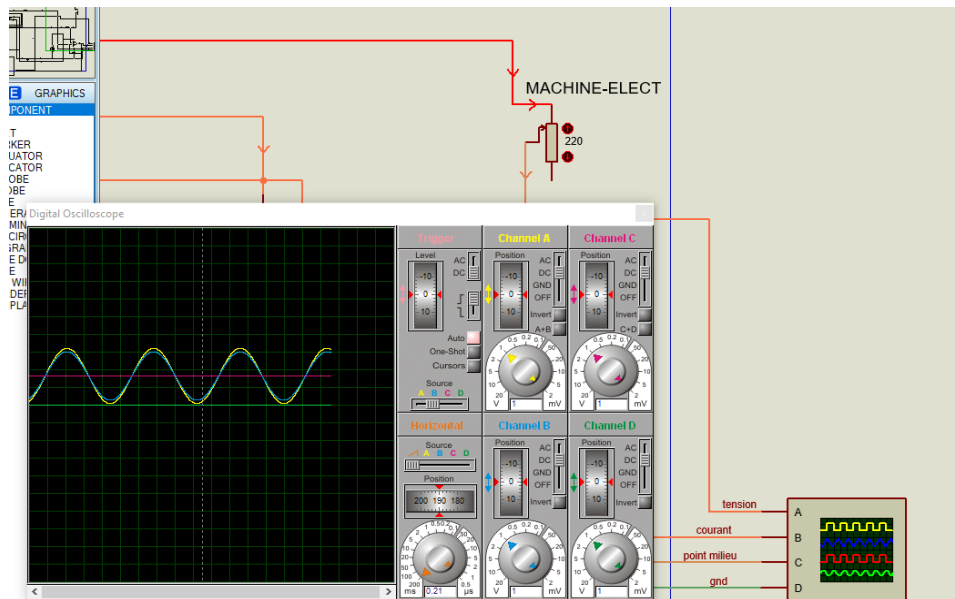


Figure 13 : Simulation avec faible résistance

Cette capture montre le signal de tension et de courant lorsque la charge utilisée est une résistance de faible valeur. On observe une onde de courant d'amplitude élevée, traduisant une consommation électrique importante.

$$U=220V / R_{ch}=2.20 \text{ Ohm} \rightarrow \rightarrow \rightarrow \rightarrow \rightarrow \rightarrow I = U/R = 100A$$

2 ème simulation :

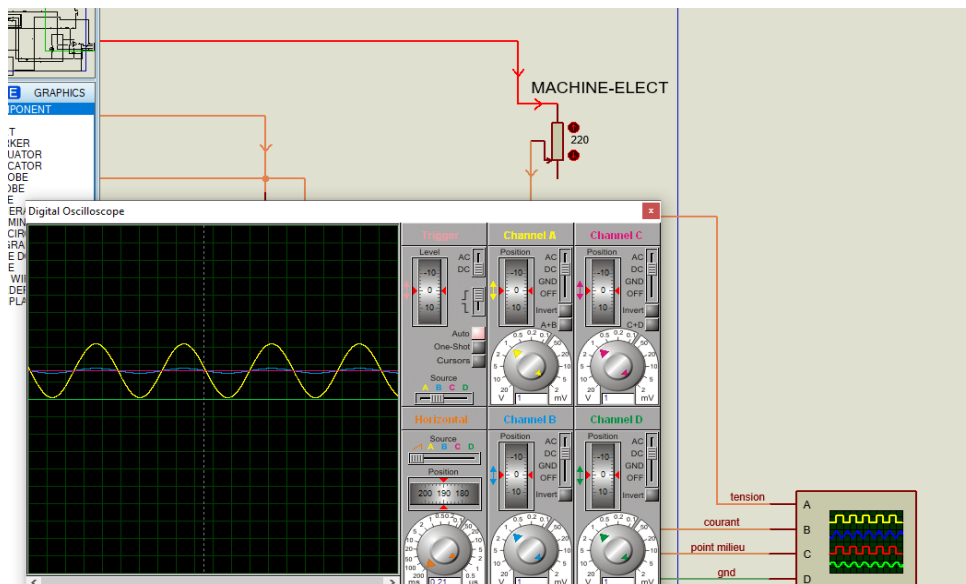


Figure 14 : Simulation avec forte résistance

Cette capture illustre le signal obtenu avec une résistance de forte valeur. Le courant mesuré présente une amplitude réduite, correspondant à une consommation plus faible de la charge connectée.

$$U=220V / R_{ch}=220 \text{ Ohm} \rightarrow \rightarrow \rightarrow \rightarrow \rightarrow \rightarrow I = U/R = 1A$$

Réalisation Matérielle

Liste des composantes

1- ESP32-C3 Super Mini :

L'ESP32-C3 Super Mini est un microcontrôleur compact et économique . Il intègre le **Wi-Fi** et le **Bluetooth Low Energy (BLE)**, ce qui le rend particulièrement adapté aux projets IoT comme le Smart Plug. Dans ce projet, il assure le rôle de **cœur du système** : mesures de tension et de courant, pilotage du relais, traitement des données et communication avec le broker MQTT (HiveMQ). Sa petite taille, sa faible consommation en font une solution idéale pour la domotique.

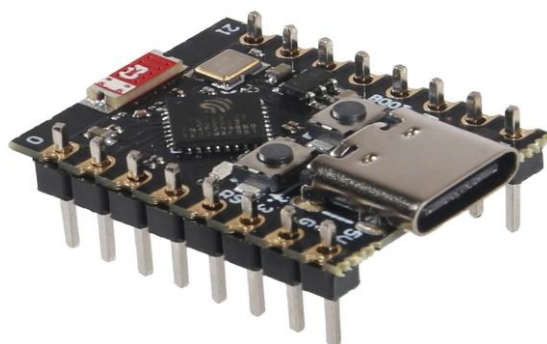


Figure 15 : ESP32 C3 Super Mini

2- Relais électromécanique :

Le relais SONGLE SRD-12V est un relais électromécanique utilisé pour commander l'alimentation de la charge connectée au Smart Plug. Alimenté par une bobine de 12 V, il permet de commuter des charges allant jusqu'à 250 V – 10 A. Il assure une isolation entre la partie commande (ESP32) et la partie puissance (220 V), garantissant ainsi la sécurité et la fiabilité du système.

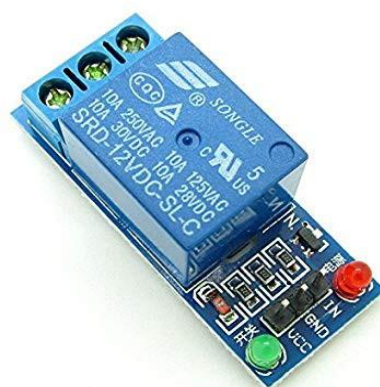


Figure 16 : Relais électromécanique

3- Convertisseur AC-DC :

Le convertisseur AC-DC est utilisé pour fournir une alimentation stable au circuit électronique ESP32-C3 du Smart Plug.

Il permet de transformer la tension secteur 220 V AC en une tension continue de 5 V ou 3,3 V, adaptée aux composants de commande.

Ce module assure la sécurité électrique en isolant la partie puissance de la partie commande.



Figure 17 : Convertisseur AC-DC

4- Capteur de tension

Pour la mesure de la tension dans mon projet de smart plug, j'ai choisi d'utiliser un simple diviseur de tension basé sur deux résistances ($R1 = 2\text{ M}\Omega$ et $R2 = 10\text{ k}\Omega$) au lieu d'un capteur dédié de type ZMPT10B.

Cette solution présente l'avantage d'être plus économique, tout en permettant d'abaisser la tension secteur à une plage sécurisée et compatible avec l'entrée analogique du microcontrôleur.

Contrairement au module ZMPT10B, qui nécessite un étalonnage précis et peut parfois générer des valeurs aléatoires en raison de son amplificateur interne et de sa sensibilité aux perturbations, le diviseur résistif offre une méthode simple, stable et fiable pour récupérer une image proportionnelle de la tension.

Bien que moins sophistiqué, ce choix est adapté à mon application, en particulier dans un contexte où la priorité est la réduction des coûts et la simplicité de mise en œuvre.

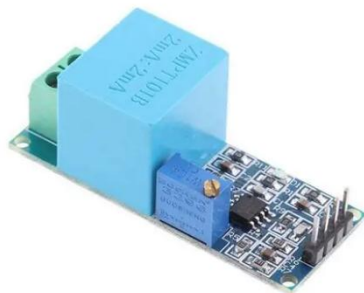


Figure 18 : Capteur de tension(ZMPT10B)

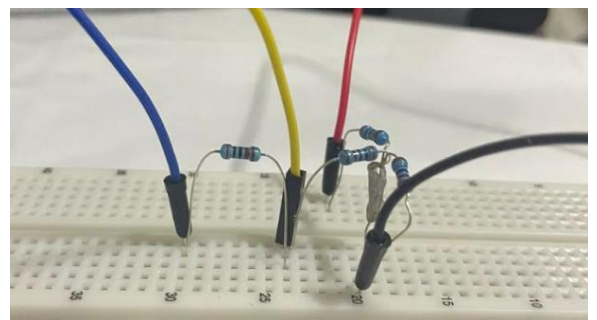


Figure 19 : Capteur de tension(Diviseur de tension)

5- Capteur de courant :

Pour la mesure du courant, j'ai opté pour la méthode du shunt résistif, en utilisant neuf résistances de $0,220\ \Omega$ montées en parallèle, placées en série avec la charge électrique. Ce montage permet d'obtenir une résistance équivalente très faible, limitant ainsi les pertes de puissance tout en générant une chute de tension proportionnelle au courant consommé.

J'ai privilégié cette approche à la place du capteur ACS712, qui présente l'inconvénient de fournir des valeurs même lorsqu'aucune charge n'est connectée et nécessite un réglage par potentiomètre pour être fiable.

La solution par shunt est non seulement plus économique, mais aussi plus simple et stable à mettre en œuvre dans le cadre de mon smart plug.

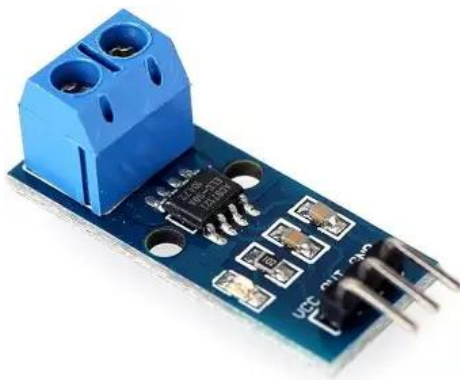


Figure 20 : Capteur de courant(ACS712)

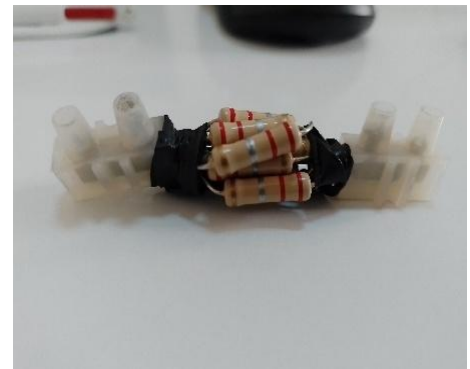


Figure 21 : Capteur de courant(Résistor Shunt)

6- Amplificateur opérationnel :

Comme la résistance shunt équivalente est très faible ($\approx 0,024\ \Omega$), la chute de tension générée reste trop petite pour être directement lue par l'ESP32.

Pour cela, j'ai ajouté un amplificateur opérationnel configuré en amplificateur différentiel, afin d'augmenter cette tension proportionnelle au courant.

Par exemple, pour un courant de 16 A, on obtient environ 0,39 V, qu'il faut amplifier avec un gain proche de 8 pour exploiter toute la plage de l'ADC (3,3 V).

L'utilisation de l'AOP assure ainsi une mesure plus précise et stable, tout en gardant la solution du shunt simple et économique.

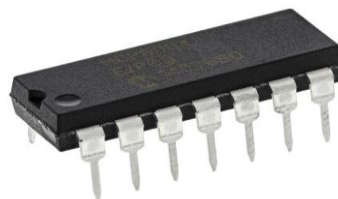


Figure 22 : Amplificateur opérationnel(MCP6004)

Montage physique

1- Câblage de la partie électrique (circuit haute puissance) :

Le circuit de haute puissance comprend la gestion de la tension secteur 220 V et la mesure du courant consommé. Le relais Songle SRD-12V est placé en série avec la charge afin de commander son alimentation.

À ce même niveau, un résistor shunt est inséré en série pour mesurer l'intensité traversant la charge. Ce composant, bien qu'exploité par la partie électronique de commande, appartient au circuit de puissance car il est directement parcouru par le courant de la charge.

Les connexions ont été réalisées avec des câbles de section adaptée et protégées par un boîtier pour garantir la sécurité de l'utilisateur.

2- Câblage de la partie électronique (circuit basse puissance) :

Le circuit de basse puissance est dédié au traitement et au contrôle du Smart Plug. Il regroupe l'ESP32-C3 Super Mini, le module d'alimentation AC-DC (qui fournit les tensions 5 V et 3,3 V), ainsi que les circuits de conditionnement des signaux.

Le résistor shunt, situé dans le circuit de puissance, envoie son signal de mesure vers un amplificateur opérationnel, qui adapte l'amplitude et ajoute un décalage de 1,65 V pour rendre le signal compatible avec l'ADC de l'ESP32. Ce dernier assure le traitement numérique, le pilotage du relais et la communication via MQTT.

L'ensemble est câblé en basse tension, isolé du secteur, garantissant sécurité et fiabilité.

Programmation et Mise en réseau

Programmation de l'ESP32

Code C++ du Smart Prise :

Le programme développé en C++ sous Arduino IDE permet d'assurer les principales fonctionnalités du Smart Plug. Tout d'abord, l'ESP32-C3 est configuré pour se connecter au réseau Wi-Fi et établir une liaison sécurisée (TLS/SSL) avec le broker HiveMQ via le protocole MQTT.

Différents topics sont définis afin d'échanger les données de mesure (courant, tension, puissance, énergie, coût) et de recevoir les ordres de commande (activation/désactivation du relais, mise à jour des seuils de sécurité, mode automatique).

La fonction principale du code repose sur l'acquisition périodique des signaux analogiques provenant du résistor shunt et du pont diviseur de tension.

Comme les valeurs mesurées par l'ESP32-C3 ne sont pas parfaitement linéaires, une **correction par régression quadratique** a été mise en place. Pour chaque échantillon, la valeur brute lue par l'ADC est normalisée, puis ajustée via une fonction quadratique :

$$y = 0.2268 * x * x + 0.7732 * x + 0.0001$$

Cette correction permet de linéariser la lecture et d'obtenir des mesures précises du courant après calcul du voltage sur le shunt et du gain appliqué.

Ensuite, les valeurs sont utilisées pour calculer le courant RMS, la tension RMS, la puissance instantanée, l'énergie consommée et le coût estimé. Ces informations sont ensuite publiées à intervalles réguliers sur le broker MQTT.

Le microcontrôleur assure également la commande du relais. Celui-ci peut être piloté soit manuellement à partir des messages MQTT reçus, soit automatiquement grâce au mode auto : dans ce cas, si le courant ou la tension dépasse les seuils fixés, le relais est coupé pour protéger l'installation.

Connexion avec HiveMQ

La **connexion avec HiveMQ** permet au smart plug de se connecter à un broker MQTT pour communiquer avec une application ou un système IoT. Grâce à cette connexion, le smart plug peut envoyer des informations comme son état (allumé/éteint) et sa consommation électrique, et recevoir des commandes à distance. La **configuration du broker MQTT** inclut la définition de l'adresse du broker, du port, des topics pour publier et s'abonner, ainsi que des paramètres de sécurité tels que l'authentification et le chiffrement SSL/TLS.

L'utilisation de **HiveMQ** présente plusieurs avantages : il offre une **communication fiable et rapide**, gère un grand nombre de connexions simultanées, et permet une **scalabilité facile** si le nombre de smart plugs augmente. HiveMQ facilite aussi la **gestion des messages en temps réel**, l'intégration avec d'autres services IoT, et peut ajouter des fonctionnalités comme le suivi de l'historique des données et la supervision à distance, ce qui améliore la performance et la sécurité du smart plug.

Interface Node Red

La **partie Interface Node-RED** concerne la création d'un **dashboard** pour visualiser et contrôler le smart plug de manière intuitive. Node-RED permet de concevoir une interface graphique où l'utilisateur peut suivre en temps réel l'état de la prise (allumée/éteinte) et sa consommation électrique, ainsi que contrôler le plug à distance via des boutons ou des interrupteurs virtuels.

La **création du dashboard** consiste à ajouter des widgets, tels que des boutons, des indicateurs et des graphiques, pour afficher les données reçues via MQTT et envoyer des commandes au smart plug. Cette interface rend l'utilisation du dispositif plus simple et interactive, tout en offrant un moyen pratique de superviser et gérer plusieurs appareils connectés à partir d'un seul tableau de bord.